

拟态语言技术手册

基础

之恪提案

2026 年 1 月 17 日

目录

1	基础概念	1
1.1	位	1
1.1.1	进制	1
1.1.2	基于进制的计数系统	1
1.1.3	存储空间中的计数系统	1
1.2	字节	2
1.3	地址	2
1.4	字长	2
2	数胞与伪代码	3
2.1	数胞	3
2.1.1	数胞的定义与表示	3
2.1.2	数胞的简写符号	3
2.1.3	数胞的抽象意义	4
2.1.4	状态数公式的再现	4
2.2	伪代码	4
2.2.1	变量与常量	5
2.2.2	赋值	5
2.2.3	函数	5
2.2.4	分支与循环	6
2.3	数胞的运算	7
2.3.1	任意进制的长除法	7
2.3.2	比较两个超出字长限制的数值	8
2.3.3	任意进制的竖式加法	9
2.3.4	任意进制的竖式减法	10
2.3.5	任意进制的竖式乘法	12

1 基础概念

1.1 位

位是计算机中表示信息的基本单位，在二进制计算机中，它可以表示 2 种状态 (0 和 1)，以此类推，在 n 进制中，它可以表示 n 种状态 (0 到 $n - 1$)。

1.1.1 进制

进制（进位计数制）是人为定义的通过基数规定进位规则的计数方法，其核心特征为“逢基数进一”。例如：

- 十进制：基数为 10，使用 0-9 共 10 个数码，逢十进一
- 二进制：基数为 2，使用 0 和 1 共 2 个数码，逢二进一
- 八进制：基数为 8，使用 0-7 共 8 个数码，逢八进一
- 十六进制：基数为 16，使用 0-9 和 A-F 共 16 个数码，逢十六进一

可以发现一个规律： n 进制逢 n 进 1，因此它的数码中不含 n 。例如十进制没有值为 10 的个位数，二进制没有值为 2 的个位数。

1.1.2 基于进制的计数系统

在日常生活中，我们使用十进制计数。在十进制的计数系统中，我们把十进制数字的各个位置分为个位、十位、百位、千位...。我们来解析一下，已知基数为 10，假设一个三位数的个位数字为 a ，十位数字为 b ，百位数字为 c 。那么它的值可以分解成 $a + 10b + 100c$ ，也就是 1 倍的 a 、10 倍的 b 、10 倍的 10 倍的 c 求和。换言之，即 $10^0a + 10^1b + 10^2c$ ，其中可以发现一个规律：如果从 0 开始计数，那么第 0 位就是个位，权重为 $10^0 = 1$ ；第 1 位就是十位，权重为 $10^1 = 10$ ；第 2 位就是百位，权重为 $10^2 = 100$ 。

现在把视角换为二进制，二进制计数系统的基数为 2，因此第 0 位的权重为 $2^0 = 1$ ；第 1 位的权重为 $2^1 = 2$ ；第 2 位的权重为 $2^2 = 4$...

现在你应该可以理解，对于任意的 n 进制，它的计数系统是如何运作的了。

1.1.3 存储空间中的计数系统

在二进制计算机的存储空间中，一个位可以表示 0 和 1，可以发现它和二进制计数系统的位的表示范围相同，因此，我们可以用计算机中的位代表计数系统中的位。例如，假设我们可以控制计算机中的 n 个位，那么我们就可以通过编码这 n 个位来表示一个 n 位的二进制数。通常，这 n 个位在计算机中是顺序存储的，你可以把位想象为一个小方格，它们在存储空间中是紧挨着的，排成一个长条。

现在你应该可以理解，计算机是如何表示数字的了。

1.2 字节

字节是计算机中由多个位组成的一个单位，例如，在目前的计算机中，我们规定 8 个位组成一个字节。假设进制为 n ，一个字节由 m 个位组成，那么一个字节的位数为 n^m ，表示范围就是 $[0, n^m - 1]$ 。

1.3 地址

地址是一个数字，每个地址都代表了计算机中某一个字节的位置。例如，地址 n 和 $n + 1$ 指向两个在存储空间中相邻的字节。

因此，如果我们要准确地在计算机中表示一个唯一的位的位置，可以使用这个位所在的字节的地址，以及它在这个字节中是第几个位来表示。

1.4 字长

字长是衡量计算机性能的一个关键指标，它决定了计算机一次操作所能处理的数据量。例如，64 位且字节为 8 位的计算机的字长就是 8 个字节，它决定了这台计算机一次操作可以处理 8 字节的数据。

因此，此计算机中大部分的操作至少都可以一次性处理 8 个字节，例如加减乘除等。

2 数胞与伪代码

2.1 数胞

在前面的章节中，我们学习了比特、字节等存储单元。我们发现，无论是哪种进制的计算机，其存储单元都可以被概括为两个核心属性：

1. 该单元所能呈现的**所有可能状态的数量** (n)。
2. 该单元在某一时刻所处的**具体状态值** (m)。

为了用一种统一、简洁的数学语言来描述计算机中所有的存储空间，我们引入了“数胞” (Numerical Cell) 这一抽象概念。它将帮助我们跳出二进制、十进制等具体进制的限制，从更高层面理解存储的本质。

2.1.1 数胞的定义与表示

一个数胞 (NC) 可以由一个二元组完整定义：

$$NC \equiv (n, m)$$

其中：

- n ：表示该数胞的**状态数** (Number of States)。它一般是一个大于 1 的自然数，决定了该存储单元能表示多少种不同的情况。它本质上对应了前文所述的“进制”概念。
- m ：表示该数胞的**值** (Value)。它是该存储单元在当前时刻的具体状态，是一个自然数，且必须满足 $0 \leq m < n$ 。

2.1.2 数胞的简写符号

为了书写方便，我们定义了一套简写符号：

- NC ：数胞 (Numerical Cell) 的通用缩写。
- NC_n ：表示所有状态数为 n 的同一类型数胞的集合。它强调了存储单元的“类型”或“容量”。
- $NC_n(m)$ ：表示一个状态数为 n ，且当前值为 m 的特定数胞。这是最完整的表示形式，同时包含了存储单元的属性 and 当前状态。

示例：

- $NC_2(1)$ ：表示一个状态数为 2 (即二进制)、当前值为 1 的数胞。这其实就是我们熟悉的一个比特 (Bit)，其值为 1。

- $NC_{256}(65)$: 表示一个状态数为 256、当前值为 65 的数胞。这恰好可以表示一个值为 65 的字节 (Byte)，因为一个 8 位二进制字节有 $2^8 = 256$ 种状态。
- $NC_{10}(7)$: 表示一个状态数为 10 (即十进制)、当前值为 7 的数胞。这可以想象为一个十进制的基本存储单元。

2.1.3 数胞的抽象意义

“计算机中所有的存储空间都可以抽象为一个数胞。”这句话的含义是：无论存储空间的实际物理实现如何，我们都可以用数胞的二元组模型 (n, m) 来刻画它。

- 若一个存储单位可以表示 N 个状态：这意味着它的“类型”是 NC_N 。
- 且其当前值为 M ：这意味着它的当前状态是 M ，且 $0 \leq M < N$ 。
- 那么它可以被数胞 $NC_N(M)$ 表示：这个数胞的表示完全捕获了该存储单元的核心信息。

数胞的抽象威力在于其通用性。它不关心底层是二进制电路、三进制器件还是其他任何物理实现，它只关心逻辑上的状态数量和一个具体的状态值。这使得我们可以在统一的框架下讨论不同架构的计算机存储问题。

2.1.4 状态数公式的再现

一个由 k 个 NC_n 类型的数胞连续组成的存储空间，其总状态数正是我们熟悉的公式：

$$\text{总状态数} = n^k$$

这个公式解释了为什么一个由 8 个 NC_2 (比特) 组成的字节 ($n = 2, k = 8$) 有 256 种状态 (2^8)，也解释了一个由 2 个 NC_{10} (十进制单元) 组成的存储空间有 100 种状态 (10^2)。

2.2 伪代码

伪代码是一种类似自然语言和编程语言的混合描述方式，用于表达算法逻辑而不受具体编程语法限制。它像是算法的“蓝图”，帮助我们在编写实际代码前理清思路。

为什么使用伪代码？

- 语言无关：无论使用什么编程语言，伪代码都适用。
- 专注于逻辑：避免被具体语法细节干扰。
- 易于理解：即使是初学者也能看懂基本逻辑。
- 便于协作：团队讨论算法时可作为“通用”语言。

之后的内容或多或少都有可能使用伪代码描述具体算法。

2.2.1 变量与常量

变量是存储数据的容器，其值可以在程序运行过程中改变。你可以把变量想象成一个贴了标签的盒子，盒子里可以放不同的东西，但标签名称保持不变。在硬件层面，变量是一块被标记了的存储空间，存储具体的值。在软件层面，变量有一个名称，使用这个名称可以访问到其代表的存储空间，并可以读取或写入这块存储空间。

常量是值，或者是不会改变的命名数据。一旦定义，其值在整个程序运行期间保持不变。在伪代码中，我们可以说明一个名称代表常量，那么之后就不能再直接修改它的值。

虽然伪代码不严格要求数据类型，但了解常见类型有助于设计更好的算法。例如，我们可以说明变量

$$x : \text{自然数}$$

是自然数，说明常量

$$y : \text{整数} := 0$$

是一个为 0 的整数。在这里，“:=”表示定义符号，通常用于声明一个常量。

2.2.2 赋值

赋值是计算机语言中几乎最常见的操作。我们规定使用左箭头表示赋值，例如：

$$x \leftarrow 3$$

表示把 3 这个值赋给 x 这个变量，在这条代码执行之后，直到 x 的值再次被改变之前， x 存储的值将保持为 3。

如果赋值的对象是常量，我们规定这是错误的写法，例如：

$$5 \leftarrow 3$$

或者，假设存在常量 $\alpha = 5$ ：

$$\alpha \leftarrow 3$$

这两种都是错误的写法，显然，一个数字不能被直接赋予另一个数字的值。

2.2.3 函数

函数是一些代码或功能的封装与抽象。我们可以定义函数：

$$f(x_1, x_2, \dots, x_n)$$

其中 f 是函数的名称， $x_1, x_2, \dots, x_n$ 是函数的 n 个参数， n 可以是任意自然数。

在定义了函数之后，可以使用它的名称和实际的参数获取它的结果，例如：

$$f(x) := x + x$$

$$y \leftarrow f(2)$$

那么在执行过后，有 $y = 4$ 。

在计算过程中，如果函数除了返回值外还会改变程序的状态（例如修改全局变量、进行输入输出等），则称该函数具有副作用。与带副作用的函数相对，函数式风格强调使用纯函数：函数的输出仅由输入决定，且不产生任何副作用。

我们规定，对于一个纯函数 $g(x, y) := x + y$ ，如果这样使用：

$$z \leftarrow g(1)$$

那么 z 这个变量是一个函数，它输入一个参数 y ，输出 $1 + y$ 。可以视为未完成的 g 函数，等待之后的参数输入，再输出最终结果。

2.2.4 分支与循环

编程语言通常有三种执行结构，顺序结构、分支结构、循环结构。

顺序结构，顾名思义，将 n 条代码（例如赋值语句）以顺序的方式一条条写下来，其执行过程就是顺序执行，例如：

$$\begin{aligned} x &\leftarrow 2 \\ y &\leftarrow x + 6 \\ z &\leftarrow y/x \end{aligned}$$

重点在于分支结构和循环结构。

分支结构中有多个分支，每个分支可以是数句代码，我们通常可以使用这样的语法结构表示：

$$\begin{array}{ll} \text{如果 } C \text{ 则} & A \\ \text{否则} & B \end{array}$$

其中， C 是一个条件，它的结果是一个布尔值，即真或假。如果为真，则执行 A ，不执行 B ；如果为假，则执行 B ，不执行 A 。

循环结构有多种表示方式，例如当型循环和直到型循环。

当型循环的结构如下：

$$\text{当 } C \text{ 则 } A$$

直到型循环的结构如下：

$$\text{执行 } A \text{ 直到 } C$$

其中 C 是条件， A 是循环体，即其他代码，区别在于：当型循环先判断是否要进入函数体，再执行；直到型循环先执行函数体，再判断是否要回过头继续执行循环体。

这三种结构可以互相嵌套，例如，循环结构内可以是分支结构，也可以是另一个循环结构。

2.3 数胞的运算

在数胞的运算中，我们默认互相运算的两个数胞，其状态数是相同的。

无论是什么运算，其结果若可能超出状态数的限制，即数胞的值溢出，则需要通过取模限制在状态数内。例如数胞的加法：

$$a : NC_n + b : NC_n \equiv NC_n((a \text{ 的值} + b \text{ 的值}) \bmod n)$$

减法、乘法、除法（整除）、取模同理，只是把上述表达式的加法符号替换掉。只不过由于除法和取模的结果不会导致数胞的值变得更大，且结果恒大于等于 0，因此一个数胞经过除法和取模运算之后，可以省略 $\bmod n$ 这个步骤。

2.3.1 任意进制的长除法

在计算机中，我们常常需要处理超出单个数胞表示范围的大数。虽然现代编程语言通常提供了大数运算库，但理解其底层原理——特别是如何在任意进制下进行除法运算——对于深入理解计算机算术至关重要。

长除法是一种适用于任意进制的算法，它通过迭代的方式，将复杂的除法问题分解为一系列简单的单数胞运算。这种方法的核心思想与我们在小学学习的十进制长除法完全相同，只是将其推广到了任意进制 b 。

考虑两个自然数：被除数 D 和除数 d ，其中 $d \neq 0$ 。我们的目标是找到商 Q 和余数 R ，使得：

$$D = Q \times d + R, \text{ 其中 } 0 \leq R < d$$

现在，假设我们不是在十进制下，而是在 b 进制下表示这些数字 ($b \geq 2$)。那么 D 和 d 可以表示为：

$$D = (d_{n-1}d_{n-2} \dots d_1d_0)_b = \sum_{i=0}^{n-1} d_i \times b^i$$

$$d = \text{除数在进制 } b \text{ 的表示}$$

长除法的过程就是从被除数的高位开始，逐位（或逐小组）地处理，每一步计算部分被除数除以除数的结果。

算法步骤

以下是任意进制 b 下长除法的通用算法步骤：

$R : \mathbb{N} \leftarrow D, Q : \mathbb{N} \leftarrow 0$

$i : NC_{\text{WORDSIZE}_{\text{MAX}}+1} \leftarrow 0$, 其中 $\text{WORDSIZE}_{\text{MAX}}$ 是当前计算机可表示的最大地址

当 $i < n$ 则

$\text{sub} : \mathbb{N} \leftarrow R$ 从最高位到 $n-1$ 位 (从 0 开始计数) 的数字所表示的值

如果 $\text{sub} \geq d$ 则令 R 对应于 sub 的部分更新为减去 d 所剩的值,

$Q \leftarrow Q + \text{sub}$ 向下取 d 的倍数 $\times b^{n-1-i}$

$i \leftarrow i + 1$

执行完成后, Q 就是长除法得到的商, R 就是长除法得到的余数。

对于令 R 对应于 sub 的部分更新为减去 d 所剩的值, 为了方便理解, 我们使用一个十进制下的例子。假设当前 $R = 12368$, sub 为最高位到第二位 (从 0 开始计数), 即 123, 且 $d = 106$, 那么更新过后, $R = 1768$, 因为 $123 - 106 = 17$ 。

对于 $Q \leftarrow Q + \text{sub}$ 向下取 d 的倍数 $\times b^{n-1-i}$, sub 向下取 d 的倍数 即为 Q 在第 $n-1-i$ 位的商。我们依旧使用一个十进制下的例子。假设当前 $R = 4478$, sub 为最高位到第二位 (从 0 开始计数), 即 44, 且 $d = 10$, 那么 Q 在第二位的商为 sub 整除 $d = 4$ 。至于 $\times b^{n-1-i}$ 这个操作, 是起到将 Q 在第 $n-1-i$ 位的商移动到第 $n-1-i$ 位的作用。

此算法展示了如何通过迭代和模运算, 将复杂的大数除法分解为一系列简单的、可在单个数胞上完成的操作。这正是计算机处理超出其原生字长大数的核心思想: 通过合适的算法和数据结构, 用多个小单元协作完成大任务。

这个算法虽然简单, 但却是理解计算机如何处理大数运算的基础。通过将复杂问题分解为可管理的步骤, 我们能够在有限的硬件资源上实现任意精度的算术运算。

任意状态数的数胞的除法和取模可以通过任意进制的长除法模拟。

2.3.2 比较两个超出字长限制的数值

假设 $\text{WORDSIZE}_{\text{MAX}}$ 是当前计算机可表示的最大地址, 即一字长存储空间可表示的最大值, 那么, 如果要表示大于 $\text{WORDSIZE}_{\text{MAX}}$ 的值, 就必须使用多个字长的存储空间。由于计算机一次只能对一字长存储空间的值进行比较 (大于、小于、等于、大于等于、小于等于、不等于), 因此, 对于这种过大的数值, 需要使用特定算法逐位比较, 其中每一位是一个一字长存储空间。

算法步骤

我们设 $\text{bit}(x : \mathbb{N}, \text{pos} : \mathbb{N}) \rightarrow NC_{\text{WORDSIZE}_{\text{MAX}}+1}$ 输入一个任意大小的数值 x (可能使用多个字长的存储空间, 这里用自然数表示), 输出 x 在第 pos 位的数值 (从 0 开始计数)。

其等价数学表达式为：

$$\text{bit}(x : \mathbb{N}, \text{pos} : \mathbb{N}) \rightarrow NC_{\text{WORDSIZE}_{\text{MAX}}+1} = NC_{\text{WORDSIZE}_{\text{MAX}}+1} \left(\frac{x}{b^{\text{pos}}} \bmod b \right)$$

其中 b 为计算机进制。

设 $\text{length}(x : \mathbb{N}) \rightarrow \mathbb{N}$ 输入一个任意大小的数值 x ，输出 x 的有效位数。其等价数学表达式为：

$$\text{length}(x : \mathbb{N}) \rightarrow \mathbb{N} = \begin{cases} 1, & \text{如果 } x = 0 \\ \log_b(x) + 1, & \text{否则} \end{cases}$$

其中 b 为计算机进制。

设要比较的两个数值为 $A : \mathbb{N}$ 和 $B : \mathbb{N}$ 。具体的算法步骤如下：

如果 $\text{length}(A) < \text{length}(B)$ 则输出A小于B

如果 $\text{length}(A) > \text{length}(B)$ 则输出A大于B

否则

$i : \mathbb{Z} \leftarrow \text{length}(A) - 1$ ■此时 $\text{length}(A) = \text{length}(B)$

当 $i \geq 0$ 则

如果 $\text{bit}(A, i) < \text{bit}(B, i)$ 则输出A小于B

如果 $\text{bit}(A, i) > \text{bit}(B, i)$ 则输出A大于B

$i \leftarrow i - 1$

输出A等于B

这个算法的核心思想是，有效位相对更长的数值更大，有效位一样长时，高位的数字相对大就大，高位的数字相对小就小。

2.3.3 任意进制的竖式加法

竖式加法的核心思想与十进制竖式加法完全相同：将两个数的各位数字对齐，从最低位（最右边）开始逐位相加，并将产生的进位传递到更高位。此方法可以自然地推广到任意进制 $b(b \geq 2)$ 。

考虑两个自然数 A 和 B ，它们在进制 b 下的表示分别为：

$$A = (a_{m-1}a_{m-2} \dots a_1a_0)_b = \sum_{i=0}^{m-1} a_i \times b^i$$

$$B = (b_{n-1}b_{n-2} \dots b_1b_0)_b = \sum_{i=0}^{n-1} b_i \times b^i$$

其中， $0 \leq a_i < b, 0 \leq b_i < b$ 。

我们的目标是计算它们的和 $S = A + B$ ，并同样用进制 b 表示结果：

$$S = (s_{k-1}s_{k-2} \dots s_1s_0)_b$$

其中， $k = \max(m, n) + 1$ (考虑最高位可能的进位)。

算法步骤

```

carry ← 0   ■存储进位
i ← 0
■逐位相加
当 i < k 则
    ■获取当前位的数字 (若该位不存在则视为 0)
    digitA ←  $\begin{cases} a_i, \text{ 如果 } i < m \\ 0, \text{ 否则} \end{cases}$ 
    digitB ←  $\begin{cases} b_i, \text{ 如果 } i < n \\ 0, \text{ 否则} \end{cases}$ 
    ■计算当前位的和 (包括低位来的进位)
    sum_temp ← digitA + digitB + carry
    ■计算当前结果位的值和新进位
    si ← sum_temp mod b
    carry ←  $\lfloor \frac{\text{sum\_temp}}{b} \rfloor$    ■ $\lfloor x \rfloor$ 指对x向下取整
    i ← i + 1
得到最终结果S, 其数值等于A + B

```

任意状态数的数胞的加法可通过任意进制的竖式加法配合限制在状态数内的手段模拟。

2.3.4 任意进制的竖式减法

竖式减法的核心思想与十进制竖式减法完全相同：将两个数的各位数字对齐，从最低位 (最右边) 开始逐位相减，并在需要时向高位借位。此方法可以自然地推广到任意进制 $b (b \geq 2)$ 。

考虑两个自然数 A 和 B ，且 $A \geq B$ (确保结果非负)。它们在进制 b 下的表示分别为：

$$A = (a_{m-1}a_{m-2} \dots a_1a_0)_b = \sum_{i=0}^{m-1} a_i \times b^i$$

$$B = (b_{n-1}b_{n-2} \dots b_1b_0)_b = \sum_{i=0}^{n-1} b_i \times b^i$$

其中， $0 \leq a_i < b, 0 \leq b_i < b$ 。

我们的目标是计算它们的差 $D = A - B$ ，并同样用进制 b 表示结果：

$$D = (d_{k-1}d_{k-2} \dots d_1d_0)_b$$

其中， $k \leq \max(m, n)$ (结果的位数可能小于原数的位数)。

算法步骤

$\text{borrow} \leftarrow 0$ ■存储借位

$i \leftarrow 0$

■逐位相减

当 $i < \max(m, n) - 1$ 则

■获取当前位的数字 (若该位不存在则视为 0)

$$\text{digitA} \leftarrow \begin{cases} a_i, & \text{如果 } i < m \\ 0, & \text{否则} \end{cases}$$

$$\text{digitB} \leftarrow \begin{cases} b_i, & \text{如果 } i < n \\ 0, & \text{否则} \end{cases}$$

■计算当前位的差 (包括低位来的借位)

$$\text{diff_temp} \leftarrow \text{digitA} - \text{digitB} - \text{borrow}$$

■计算当前结果位的值和新借位

$$d_i \leftarrow \text{diff_temp} \bmod b$$

$$\text{borrow} \leftarrow \text{如果 } \text{diff_temp} < 0 \text{ 则为 } 1 \text{ 否则为 } 0$$

$$i \leftarrow i + 1$$

得到最终结果 D , 其数值等于 $A - B$

任意状态数的数胞的减法可通过任意进制的竖式减法配合限制在状态数内的手段模拟。

2.3.5 任意进制的竖式乘法

竖式乘法的核心思想与十进制竖式乘法相同：将一个乘数的每一位与另一个乘数的每一位相乘，然后将结果按位对齐并相加。此方法可以自然地推广到任意进制 $b(b \geq 2)$ 。

考虑两个自然数 A 和 B ，它们在进制 b 下的表示分别为：

$$A = (a_{m-1}a_{m-2} \dots a_1a_0)_b = \sum_{i=0}^{m-1} a_i \times b^i$$

$$B = (b_{n-1}b_{n-2} \dots b_1b_0)_b = \sum_{j=0}^{n-1} b_j \times b^j$$

其中， $0 \leq a_i < b, 0 \leq b_i < b$ 。

我们的目标是计算它们的乘积 $P = A \times B$ ，并同样用进制 b 表示结果：

$$P = (p_{k-1}p_{k-2} \dots p_1p_0)_b$$

其中， $k \leq m + n$ (乘积的位数最多为两个乘数位数之和)。

算法步骤

$carry \leftarrow 0$ ■存储进位

$i \leftarrow 0$

■逐位相乘

设 $T = (t_{k-1}t_{k-2} \dots t_1t_0)_b, T \leftarrow 0$ ■存储临时中间结果

当 $i < n$ 则

■获取B当前位的数字 (若该位不存在则视为 0)

$$\text{digitB} \leftarrow \begin{cases} b_i, & \text{如果 } i < n \\ 0, & \text{否则} \end{cases}$$

$j \leftarrow 0$

当 $j < m$ 则

■获取A当前位的数字 (若该位不存在则视为 0)

$$\text{digitA} \leftarrow \begin{cases} a_j, & \text{如果 } j < m \\ 0, & \text{否则} \end{cases}$$

■计算乘积 (包括低位来的进位)

$\text{prod_temp} \leftarrow \text{digitA} \times \text{digitB} + \text{carry}$

■计算当前临时位的值和新进位

$t_{i+j} \leftarrow t_{i+j} + \text{prod_temp} \bmod b$

$\text{carry} \leftarrow \lfloor \frac{\text{prod_temp}}{b} \rfloor$ ■ $\lfloor x \rfloor$ 指对x向下取整

$j \leftarrow j + 1$

■如果最后一次乘法依然产生进位, 将其附加在专门为进位预留的最后一位上

如果 $\text{carry} > 0$ 则 $t_{i+j} \leftarrow \text{carry}$

$i \leftarrow i + 1$

$P \leftarrow T$

得到最终结果P, 其数值等于 $A \times B$

任意状态数的数胞的乘法可通过任意进制的竖式乘法配合限制在状态数内的手段模拟。

需要注意的是, 竖式乘法的时间复杂度为 $O(n^2)$ 。对于非常大的数, 可以使用更高效的算法来优化性能, 但竖式乘法作为最基础和最直观的算法, 仍然是理解乘法原理的重要工具。